

Weekly Project Progress Report

Team Name: [MedCare Team](#)

Week Number: [Week10](#)

Team Members: [Aya Ali](#), [Mlaak Al-Shawa](#), [Lama Alhaj](#)

Work completed this week

This week, we developed the core server-side backend logic using **Pure PHP** and **SQLite** database engine. We focused on building clean database connection routines and dividing our operations into modular helper functions to manage records. We also implemented role-based session checking and route guards to ensure the user roles (Patient, Doctor, Admin) remain separated, and crafted a polished, glassmorphism theme using responsive vanilla CSS.

Features implemented

We implemented the backend CRUD functionality for our MedCare System. We created the database initializer (**init_db.php**) to automatically generate the schema and seed custom credentials with secure hashed passwords. On the patient dashboard, we added specialty filtering and query-based search. We created **my-appointments.php** which loads dynamic logs and supports cancelling (removing from DOM) and rescheduling appointments through interactive modal forms. On the doctor portal, we added a status management control system and profile availability configs. On the administrative side, we implemented tabs to view, insert, modify, and delete doctor and patient profiles, successfully satisfying all CRUD requirements (BookAppointment, AdminDashboard, DoctorDashboard, Login, Register, MyAppointment, and PatientDashboard).

Problems encountered

We faced issues handling transactional consistency during multi-table writes, where a new user registration must atomically initialize a corresponding doctor or patient profile. We also faced session synchronization lags where updating doctor or patient names from their settings didn't immediately update the header display name without session refreshes. Finally, we resolved duplicate unique key constraints on seeded emails during our initial script setups.

Solutions applied

To fix the transaction problem, we utilized PDO transaction support (**beginTransaction**, **commit**, and **rollBack**) to guarantee atomic queries. We resolved the session sync lag by directly updates to

`$_SESSION['full_name']` in our model update functions on success. Duplicate email conflicts were solved by standardizing patient and doctor seed credentials so that every account represents a distinct email.

Plan for next week

On the next week (Week 11), we will focus on the **Integration** side (Frontend + Backend + DB) to assemble all these components into a single system. We plan to verify that data flows correctly into the frontend HTML layouts, ensure all navigation links and dynamic routes function correctly, and perform end-to-end integration tests.

Questions for instructor

Should we prepare to deploy this Pure PHP and SQLite database setup to a production staging server for validation, or is local hosting sufficient for review?

❖ Project Code and Repository:

- The project source code was organized into multiple PHP pages and modules to keep the backend structure clean and maintainable
- Each page handles a specific functionality such as authentication, appointment booking, and appointment management

الاسم	النوع	الحجم المضغوط	محمي بك
config	مجلد ملفات		
css	مجلد ملفات		
db	مجلد ملفات		
includes	مجلد ملفات		
models	مجلد ملفات		
admin-dashboard.php	PHP Source File	4 كيلوبايت	لا
book-appointment.php	PHP Source File	2 كيلوبايت	لا
doctor-dashboard.php	PHP Source File	3 كيلوبايت	لا
index.php	PHP Source File	1 كيلوبايت	لا
init_db.php	PHP Source File	2 كيلوبايت	لا
login.php	PHP Source File	2 كيلوبايت	لا
logout.php	PHP Source File	1 كيلوبايت	لا
my-appointments.php	PHP Source File	3 كيلوبايت	لا
patient-dashboard.php	PHP Source File	2 كيلوبايت	لا
register.php	PHP Source File	2 كيلوبايت	لا

- **GitHub Repository:**

https://github.com/LamaAlhaj22/MedCare_System

- **Main Implemented Pages and Functionalities:**

- ➔ Login
- ➔ Register
- ➔ Patient Dashboard
- ➔ Doctor Dashboard
- ➔ Admin Dashboard
- ➔ Book Appointment
- ➔ My Appointments
- ➔ Logout